Project title: Smart Transportation System

Full name of student: Joshua Anto

Student ID: 17988440

Project Supervisor: Hakilo Sabit
Academic Supervisor: Anubha Karla

Statement of Originality

I hereby declare that this submission is my own work and that, to the best of my knowledge and belief, it contains no material previously published or written by another person except where explicitly defined, nor material which to a substantial extent has been submitted for the award of any other degree or diploma of a university or other institution of higher learning.

X 

Joshua Anto

16/10/2024

Date

Acknowledgements

I would like to express my heartfelt gratitude to my parents for their unwavering support and encouragement, which has provided me with the opportunity to pursue my studies and complete this project. I am also deeply thankful to my project supervisor, whose guidance, expertise, and support have been invaluable throughout the journey of completing my final year project. Additionally, I would like to extend my sincere appreciation to the academic supervisor of this course for their efforts in organizing and supporting students, ensuring we had the necessary resources and guidance to succeed.

Abstract

This report explores the development and application of a smart transportation system, focusing on adaptive traffic light control for improving traffic flow and intersection safety. Using the SUMO simulation environment and the TRACI Python interface, the project aimed to manage traffic light phases based on real-time data from vehicle and pedestrian detectors, with the long-term goal of preparing for the integration of autonomous vehicles into New Zealand's transport infrastructure.

Key objectives included designing and modelling traffic flow for the Symonds Street area, developing a dynamic traffic light management system, and gaining insights into how autonomous vehicles could interact with existing traffic systems. The project successfully met its objectives by creating a system capable of adapting to real-time demands, demonstrating improvements in traffic efficiency and pedestrian safety. However, challenges such as integrating pedestrian and vehicle detection in a single script and refining traffic light control logic indicate that further work is needed to optimize the system.

Future work will focus on expanding the simulation to more complex intersections, integrating additional transport modes like buses and emergency vehicles, and employing AI to enhance traffic responsiveness. The project sets the groundwork for future smart transportation systems, offering valuable insights into real-time traffic management and the future role of autonomous vehicles in urban environments.

Table of Contents

List of Figures

List of Tables

Chapter 1: Introduction

Background Information

A smart transportation system, also known as an Intelligent Transportation System (ITS), is a sophisticated and integrated method of managing and optimising many aspects of transportation using technology and data. A smart transportation system's major purpose is to increase the efficiency, safety, and sustainability of transportation networks while improving user mobility. Within this smart transportation system, we will mostly be focusing on the smart traffic light system within an intersection. Traditional transportation systems rely significantly on the processes that activate traffic lights to coordinate traffic flow. Timers, pressure plates implanted beneath the road surface, and pedestrian buttons placed on the kerbside are examples of these triggers.

Drivers and pedestrians are both responsible for adhering to and obeying traffic signals. These signals are critical communication tools that help to ensure safe and efficient road travel. When any of these components fails, the efficiency and safety of the overall system are jeopardised. For example, a driver's brief lack in concentration could result in their running a red light, perhaps causing an accident or impeding traffic flow. Similarly, if a pedestrian fails to hit the crosswalk button, they may miss their assigned crossing time and thus must wait longer, which can lead to frustration and perhaps hazardous behaviour. Furthermore, the traffic signals themselves are prone to failure. Consider a scenario in which a traffic light fails to turn green despite the lack of heavy traffic, leaving a single automobile stuck at a crossroads. This not only causes the motorist excessive delays, but it also has an impact on the overall flow of traffic in the area, perhaps producing congestion. An example of a smart transportation system is the GLIDE system implemented in Singapore. Singapore was the first to install a computerised urban traffic control system based on fixed time plans in 1981. In 1988, this was replaced by a dynamic computerised urban traffic control system known locally as the GLIDE system. This dynamic system serves three purposes. These include allocating optimal green time to each approach at a junction based on the volume of traffic on each of these approaches, providing linkage for nearby traffic lights, and centrally monitoring the state of the traffic signals in the system. The system can operate in one of three modes.

The system normally operates in Masterlink mode, which automates the optimal allocation of cycle times, green times, and traffic signal offsets. The other two modes, Flexilink and Local, are fallback choices that are employed in the event of a telecommunications system, computer, or local controller failure. Higher journey speeds in the city region and reduced total fuel usage are two advantages of implementing the GLIDE system. A research was conducted by the following researchers Bilal Jan; Haleem Farman; Murad Khan; Muhammad Talha; Ikram Ud Din on designing a smart transportation system. A component of this article was the smart traffic lights, which we will be focussing on. The purpose of the smart traffic lights is to mitigate increasing traffic and congestions within a city aswell as pedestrian safety, reducing accidents. The proposed idea was interconnecting smart lights and signals with a traffic control system, which in turn can gather more information can be gathered regarding traffic patterns. The proposed architecture is to analyse big data using sophisticated algorithms and bring forward a system that can handle transportation data in real time. The mechanism consists of four different layers.

- Data collection Layer
- Communication Layer
- Data processing Layer
- Application Layer’s

Project Overview

This project is rooted in the field of civil and electrical engineering, focusing on the design, modelling, and simulation of a smart intersection control system aimed at enhancing urban mobility and safety. The primary goal is to significantly increase intersection safety while laying the groundwork for the integration of autonomous vehicles (AVs) into New Zealand's transportation network. By adopting a smart transportation system, the initiative aims to reduce accidents involving vehicles and pedestrians, ensuring safer road crossings for all users.

Motivation and Rationale

The motivation behind this project stems from the growing interest in autonomous vehicles and the need to optimize urban traffic flow. With autonomous vehicles poised to enter the New Zealand market, there is a pressing need for

infrastructure that can support their operation while also improving the overall efficiency of city transportation systems. The project's focus on optimizing traffic flow at intersections is driven by the objective of reducing congestion, minimizing wait times, and providing real-time traffic control to facilitate smooth and safe mobility for both vehicles and pedestrians.

Problem Definition and Objectives

The core problem addressed by this project is the need to develop an intersection control system that can dynamically adapt to real-time traffic conditions, ensuring optimal flow and safety. The specific objectives include:

- Designing and modelling traffic flow for a selected urban area, specifically Symonds Street from the AUT-WZ building to Anzac Avenue.
- Developing a Python-based TRACI (Traffic Control Interface) script to manage traffic light phases based on real-time data from vehicles and pedestrians at intersections.
- Simulating the traffic flow with the assumption that all vehicles are autonomous, thereby eliminating human error and providing insights into how AVs can be integrated into the existing traffic network.

Scope and Limitations

The project is scoped to include the design, modelling, and simulation of traffic flow in a defined area, with a focus on a specific set of intersections. Not all streets in the area will be included in the simulation, which is a deliberate choice to maintain a manageable scope within the project's timeframe. Assumptions include treating all vehicles as autonomous to streamline the decision-making process in the simulation and reduce the complexity associated with human-driven vehicles. Traffic data from Auckland Transport (AT) will inform the routes taken by autonomous vehicles in the simulation, ensuring realistic traffic patterns.

Success Criteria and Engineering Skills Demonstration

Success in this project will be measured by the effectiveness of the developed TRACI script in optimizing traffic light phases to reduce congestion and enhance pedestrian safety. The project will demonstrate a high level of engineering skills through the integration of real-time traffic data, the use of advanced simulation tools, and the application of programming expertise in Python. The feasibility of completing the project within the semester is ensured by the defined scope and the use of established traffic data.

Figure 1: Subject are

Chapter 2: Literature Review

The literature review focuses on the development and application of smart transportation systems, with an emphasis on the integration of Digi XBee components for real-time data exchange at smart intersections. This technology plays a pivotal role in enhancing intersection safety and optimizing traffic flow, directly addressing the project's research question concerning the current state of smart transportation systems.

Through a systematic investigation, the review synthesizes key findings from a decade of empirical studies, case studies, and simulations, evaluating Digi XBee's impact on reducing traffic congestion, improving road safety, and facilitating efficient traffic management. The literature selection process involved comprehensive searches across reputable databases such as Google Scholar, IEEE Xplore, and Scopus, using focused keywords like "smart transportation," "intersection safety," "traffic optimization," and "Digi XBee." This approach ensured that the most relevant and up-to-date research was included. Articles from 2010 to 2023 were meticulously screened, and data were extracted based on their methodologies, technological implementations, and outcomes, which helped build a clear picture of the state-of-the-art technologies in smart transportation systems.

The review correctly identifies Digi XBee as a widely adopted technology in smart transportation systems, specifically for enabling low-latency, reliable wireless communication. While advancements have been made in optimizing traffic flow and safety, several research gaps remain. Key gaps include the lack of standardized protocols for integrating smart transportation systems at larger scales, inadequate cybersecurity measures to protect these systems from potential attacks, and limited scalability solutions for expanding such systems to more urban areas. The review also highlights a need for further research into user interactions with smart technologies and long-term sustainability concerns, as these factors are critical for the widespread adoption of smart systems.

In terms of theoretical frameworks and models, the review systematically covers key technological advancements relevant to the project, including real-time data collection methods and communication protocols used in conjunction with Digi XBee components. These frameworks were cross-referenced with case studies to provide empirical evidence of their effectiveness in real-world scenarios, such as improved traffic management and enhanced pedestrian safety.

Broader contextual considerations, such as health and safety implications, ethical concerns about data privacy, equitable distribution of technology, and legal issues surrounding the use of real-time data, are also discussed. These factors are crucial for evaluating the feasibility and societal impact of deploying smart transportation systems. The review establishes a clear link between these contextual issues and the project's goals, ensuring that the project aligns with legal and ethical standards while maintaining public safety.

In conclusion, the review thoroughly identifies the current state of smart transportation technologies, while also addressing critical gaps that align with the project's aims. These include the need for standardized integration protocols, cybersecurity measures, scalable designs, and further research on user behavior and system sustainability. This comprehensive understanding of the literature not only provides a strong foundation for the project but also sets the stage for future investigations and innovations in smart transportation systems.

Chapter 3: Methodology

The process of completing the project involved several distinct phases: research, design, development and simulation.

Research Phase: This phase focused on understanding the core principles of smart transportation systems and examining how current traffic light systems operate. Through comprehensive research, we explored various transportation models and how they can be adapted to improve traffic flow and safety. The research guided our choice of software and tools, leading us to select SUMO (Simulation of Urban Mobility**)** as our primary platform for modelling and simulating traffic scenarios. Additionally, this phase emphasized the importance of programming logic, particularly with the use of Python, to control and adapt traffic lights based on real-time traffic and pedestrian demands.

Design Phase: In this phase, the conceptualization of our smart transportation system took shape. We outlined how the various components—such as Sensors, OBU's , RSU's, TLCS—would interact within a unified system. The design process involved creating a detailed blueprint for how each component would operate in relation to others, ensuring seamless communication and coordination. This provided a clear understanding of how to build the model within the chosen software and anticipate the dynamic behaviours that would occur during the simulations.

Development and Simulation Phase: This phase involved a hands-on approach to implementing the design within SUMO. We started by learning the intricacies of the SUMO environment, including how to generate traffic and pedestrian scenarios for our area of interest. To control traffic lights and implement real-time decision-making, we developed a Python script using TraCI (Traffic Control Interface**)**, which was built and tested in Notepad++ editor and SUMO environment. The TraCI script enabled us to manipulate traffic light behaviour dynamically based on traffic conditions, allowing the system to respond efficiently to both vehicle congestion and pedestrian demands. Throughout this phase, continuous iterations of simulation were conducted to fine-tune the logic and optimize system performance. This iterative process provided valuable insights for improving the model and moving closer to the goal of creating a fully functional smart transportation system.

By breaking down the project into these structured phases, we were able to systematically address each challenge and refine our system to align with real-world traffic management requirements.

Design

Detectors

A responsive and adaptable traffic management environment is produced by integrating several sophisticated sensing technologies into a smart transportation system. The detectors at the centre of this system collect data in real time about passing cars and people, allowing traffic signals to be controlled dynamically. Traffic lights can react when vehicles are waiting thanks to inductive loop detectors, which are installed in the road surface at crossings and detect the presence of vehicles through changes in magnetic fields. Infrared sensors and radar detectors, which are positioned carefully to identify pedestrians and cars and make sure that signals adjust accordingly to permit safe crossing or preserve efficient traffic flow, are a good addition to these.

To optimise signal timing, cameras fitted with advanced image processing algorithms also keep an eye on crossings, analysing the movements of cars, pedestrians, and even bicycles. By accurately detecting the presence of road users, microwave and ultrasonic sensors significantly improve detection capabilities, particularly in difficult weather conditions or surroundings. Together, these detectors and traffic light controllers process the data gathered and modify the phases of the signals in real time to accommodate the flow of traffic. The foundation of an intelligent transportation system that lowers traffic congestion, boosts safety, and guarantees a smooth and effective flow of traffic is this seamless integration of sensing technologies. For the sake of the design and simulation, inductive loop detectors have been employed to detect incoming vehicles into the junction and send the necessary data to the central control system. Other forms of detection can be incorporated in a physical model to improve accuracy and efficiency.

Communication protocol

IEEE 802.11p, a critical communication protocol in smart transportation systems, facilitates real-time data exchange between vehicles and infrastructure at intersections, enhancing safety and efficiency. Operating in the 5.9 GHz band, this protocol enables Vehicle-to-Everything (V2X) communication, allowing vehicles to communicate with traffic signals, roadside units, and other vehicles. At intersections, IEEE 802.11p ensures low-latency, reliable communication that can dynamically adjust traffic lights based on real-time traffic conditions, reducing congestion and the risk of collisions. This capability is especially crucial for managing complex traffic scenarios, where quick and accurate information exchange can significantly improve traffic flow and pedestrian safety.

Complementing IEEE 802.11p, IEEE 802.15.4 is another important communication protocol used in smart transportation systems, particularly for low-power, low-data-rate applications. Operating in the sub-GHz and 2.4 GHz frequency bands, IEEE 802.15.4 is ideal for connecting sensors, cameras, and other devices that transmit non-critical data to traffic signal controllers. For example, environmental sensors or pedestrian detection systems can use IEEE 802.15.4 to relay information about traffic volumes or pedestrian crossings to the Traffic Light Control System (TLCS) efficiently and cost-effectively. While IEEE 802.11p handles the real-time, high-priority data exchanges required for immediate traffic control, IEEE 802.15.4 enables the transmission of supplementary data that supports overall traffic management without overwhelming the network, thus optimizing the intersection's performance. Together, these protocols form the backbone of intelligent transportation systems, ensuring a balance between critical real-time decision-making and efficient, low-power data communication.

Conceptual Design

For a smart transportation system utilizing IEEE 802.11p, a variety of equipment and components are required to facilitate effective Vehicle-to-Everything (V2X) communication, especially at intersections.

- Digi XBee 802.15.4 Modules: Digi XBee 802.15.4 modules, while based on a different standard than IEEE 802.11p, can still be effectively used to fulfill certain communication needs within a smart transportation system. These 802.15.4-compatible modules are well-suited for internal communication between traffic controllers and other system components at intersections, offering reliable data transmission over short distances. In our system, these XBee modules will be embedded within traffic controllers and vehicles to gather non-critical data. Critical data transmission, such as real-time traffic updates or emergency vehicle prioritization, will be handled using IEEE 802.11p, ensuring efficient, low-latency communication between vehicles, infrastructure, and other system elements. This dual-communication approach helps balance system load while ensuring that essential traffic management tasks are executed with the highest priority.

- Sensors and Cameras: A wide array of sensors, including cameras, radar, lidar, and inductive loops, is utilized to capture real-time data on vehicle and pedestrian movements at intersections and crossings. These sensors provide crucial information for monitoring traffic flow, detecting congestion, and ensuring pedestrian safety. To transmit this data efficiently, the sensors can be linked to Roadside Units (RSUs) using IEEE 802.11p or other related communication technologies. This setup enables fast, low-latency data exchange between the sensors and the central traffic management system, allowing for dynamic traffic signal adjustments and optimized traffic flow based on real-time conditions.

- On-Board Units (OBUs): When installed in vehicles, these units facilitate communication between cars and roadside equipment, enhancing connectivity within the transportation ecosystem. Equipped with IEEE 802.11p transceivers, they enable real-time data exchange critical for effective Vehicle-to-Everything (V2X) communication. Examples include retrofit kits that add IEEE 802.11p capabilities to existing vehicles and integrated V2X communication systems designed for new models. In our system, we will utilize On-Board Units (OBUs) to establish seamless communication between vehicles and roadside units. This data can then be aggregated and transmitted to traffic light controllers for processing, enabling dynamic traffic management based on current conditions and improving overall traffic flow and safety.

- V2X Enable Devices: V2X-enabled devices for pedestrians represent an emerging frontier in road safety technology, designed to enhance awareness between pedestrians and vehicles in real time. By leveraging smartphones, wearable devices, or dedicated P2X technology, these systems can significantly mitigate the risk of accidents and facilitate better management of pedestrian traffic, particularly in complex urban environments. Functioning similarly to On-Board Units (OBUs) in vehicles, V2X devices for pedestrians enable effective communication with roadside units and traffic signal controllers. This connectivity allows for timely alerts about pedestrian presence to approaching vehicles and can prompt traffic signals to adjust in response to pedestrian movements, ultimately creating safer and more efficient urban spaces.

- Roadside Units (RSUs): Roadside Units (RSUs) are essential infrastructure elements strategically positioned at intersections to enable seamless communication between moving vehicles and traffic control systems. These units typically comprise communication processors, sensors, and transceivers that comply with IEEE 802.11p, ensuring reliable data exchange. By facilitating the transmission of critical information, RSUs play a pivotal role in enhancing traffic management. They can relay real-time data on traffic conditions, vehicle speeds, and pedestrian presence directly to traffic light controllers, allowing for dynamic adjustments to signal timings and improving overall traffic flow and safety at intersections.

- Traffic Signal Controllers: Traffic signal controllers can be enhanced by integrating V2X communication systems, allowing them to adjust signal timings based on real-time traffic data. By leveraging this technology, controllers can effectively oversee traffic light operations, dynamically responding to changing conditions on the road. They will communicate with Roadside Units (RSUs) and On-Board Units (OBUs) using IEEE 802.11p communication modules, facilitating seamless data exchange. This integration enables traffic signal controllers to optimize signal phases, reduce congestion, and improve safety by adapting to the flow of vehicles and the presence of pedestrians, ultimately creating a more efficient and responsive traffic management system.

- Centralised Traffic Management System: CTMS is an advanced, integrated platform designed to optimize and control traffic flow across a city or region in real time. By using a network of sensors, cameras, and communication devices, it collects data on vehicle movements, traffic density, and pedestrian activity at various intersections. This data is processed centrally, allowing for dynamic adjustment of traffic signal timings to reduce congestion and improve safety. Within in our system, we can embed our programming logic into a CTMS to control multiple traffic lights at intersections.

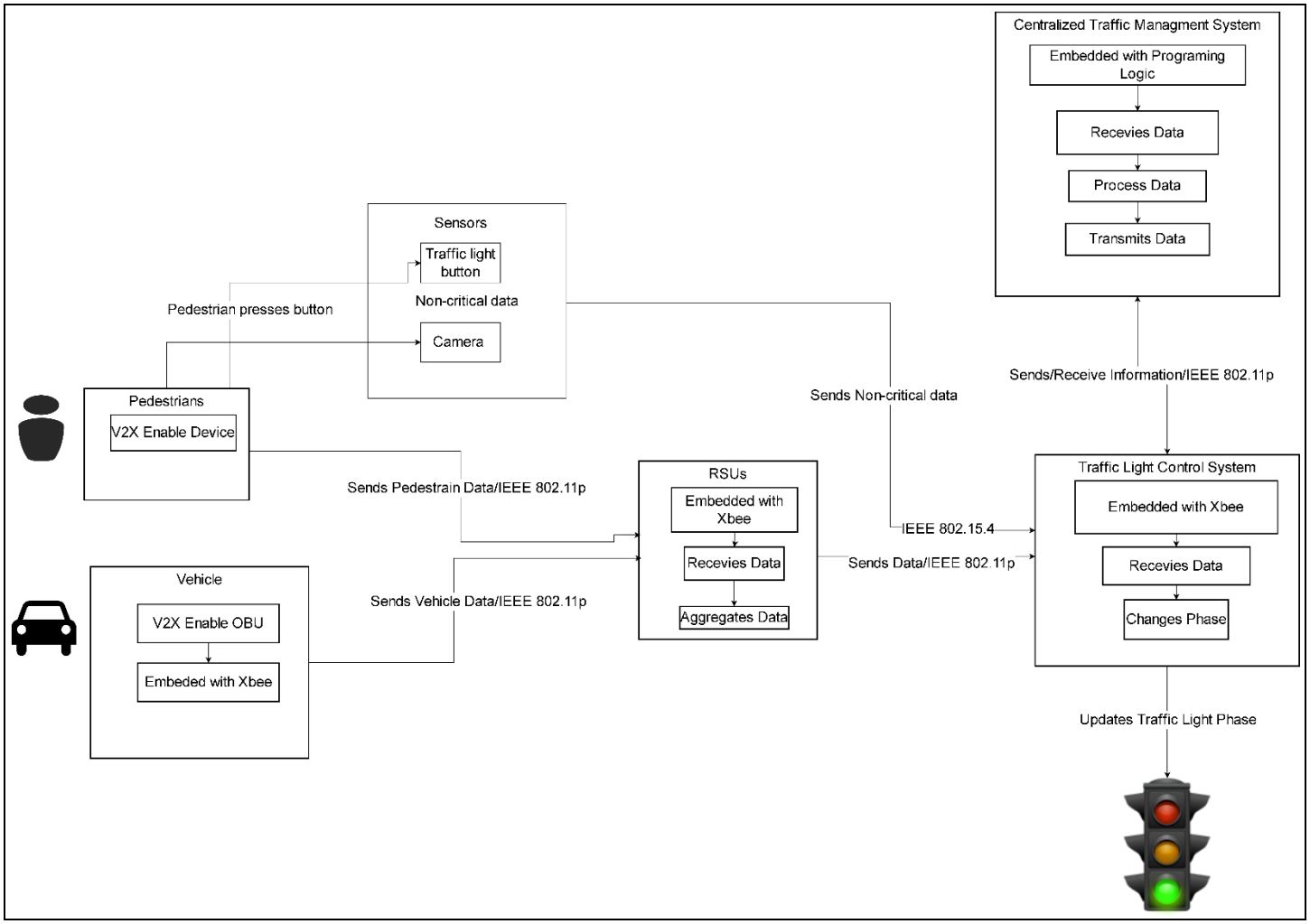


Figure 2: System Flow chart

The flowchart illustrates how an intersection operates within our system. Pedestrians with V2X-enabled devices can transmit data to roadside units (RSUs), while vehicles equipped with onboard units (OBUs) and embedded Digi XBee modules collect internal data. The vehicles then use IEEE 802.11p communication to relay this data to the RSUs. Additionally, sensors and cameras at the intersection use IEEE 802.15.4, a low-power and cost-effective communication protocol, to send non-critical data to the Traffic Light Control System (TLCS). The RSUs aggregate data from both pedestrians and vehicles, transmitting it via IEEE 802.11p to the TLCS. The TLCS manages traffic light phases based on this data; however, in our system, data is also forwarded to a Centralized Traffic Management System (CTMS). The CTMS processes data from multiple intersections and sends optimized traffic phase instructions back to each TLCS. The programming logic runs on the CTMS, where it analyses inputs from various TLCS units across different junctions to determine the optimal traffic light phase for each intersection, ensuring smooth traffic flow and pedestrian safety.

Simulation and Development

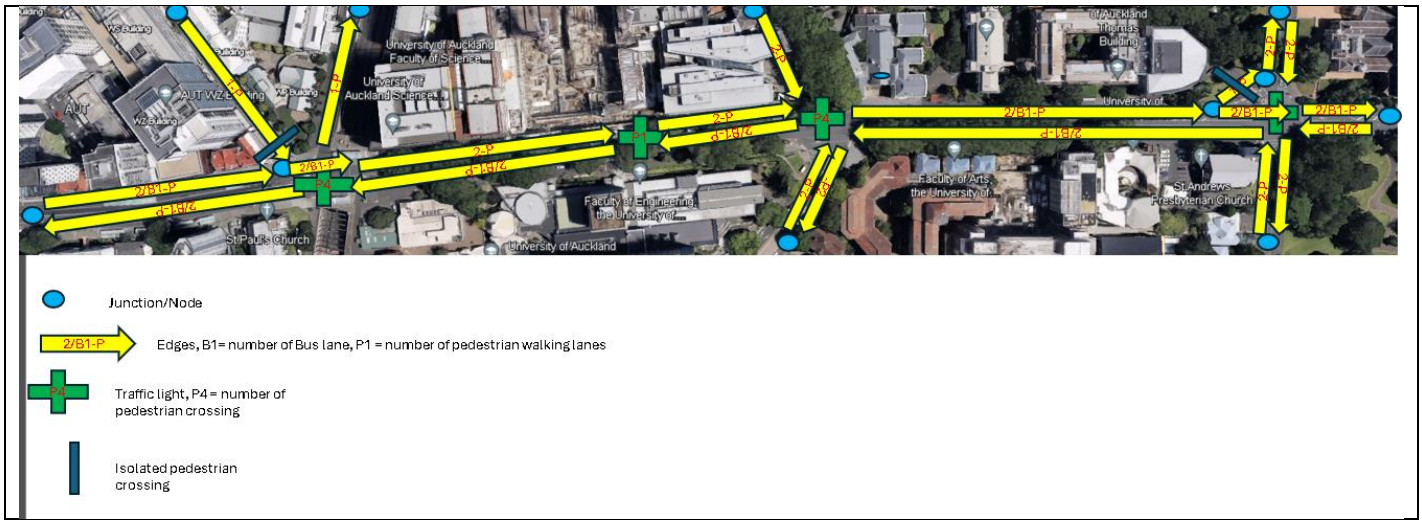
SUMO Model

The chosen software Simulation of Urban Mobility (SUMO) is an open-source, highly flexible traffic simulation tool designed for modelling complex transportation systems and analysing traffic flow in urban environments. SUMO enables users to create detailed models of road networks, incorporating real-world data and specific traffic regulations. Its versatility allows for the integration of advanced technologies such as Vehicle-to-Everything (V2X) communication, enabling the simulation of intelligent transportation systems. Additionally, SUMO supports various programming interfaces and extensions, making it an invaluable resource for studying traffic behaviour, evaluating traffic management strategies, and optimizing transportation infrastructure. Through its robust capabilities, SUMO contributes significantly to the development of smarter, more efficient urban mobility solutions.

Within SUMO, there is a design tool called NETEDIT, which plays a crucial role in building detailed traffic simulations. NETEDIT allows users to create and modify the nodes, routes, demands, and junctions within the transportation system they wish to model. This tool is particularly useful for visually constructing and adjusting various elements of the network before running traffic simulations. The initial step in our process was to identify the subject area to be modelled—in this case, Symonds Street. To accurately represent this area, we utilized Google Earth to obtain a detailed and reasonably scaled image of the site. This image was then imported into NETEDIT, providing a geographical foundation for creating an accurate layout of the roadways, intersections, and traffic signals. With this base, we were able to map out specific routes and set demands, which represent the expected traffic volumes and flows throughout the system. The flexibility of NETEDIT allowed for precise adjustments to the infrastructure, ensuring that our model closely mirrored real-world conditions. This setup enables us to simulate various traffic scenarios and test the effects of different traffic control strategies before implementing them in the real world.

Once the subject area was determined and obtained, the next step involved marking up traffic flows, intersections, and key junctions to streamline the modelling process. To do this effectively, we used PowerPoint to visually annotate the map of the subject area. This markup included highlighting the main traffic arteries, identifying critical intersections, and flagging any unique or complex junctions that required special attention in the simulation. By using PowerPoint for this step, we were able to visually organize and structure the flow of vehicles and pedestrians within the network, making the later stages of modelling in SUMO and NETEDIT much more efficient.

This pre-modelling markup acted as a blueprint, guiding how nodes, edges, and traffic demands would be configured within the simulation. Additionally, special junctions, such as pedestrian crossings or intersections with dynamic traffic lights, were highlighted to ensure that their unique operational needs were addressed during the modelling phase. This careful planning step significantly simplified the process of translating real-world conditions into a digital simulation, ensuring accuracy and ease of adjustment in subsequent simulation iterations.



After completing the markup, we proceeded to use NETEDIT to design the routes, junctions, and traffic flows within the simulation environment. This involved meticulously inputting the previously identified features, such as roads and intersections, while cross-referencing our work with both the markup and the scaled subject area image obtained from Google Earth. This cross-referencing was essential, as it helped us identify any missing elements or discrepancies between the visual plan and the digital model. By ensuring that all features aligned accurately, we maintained a high level of fidelity to the real-world conditions of Symonds Street.

The image created in NETEDIT reflects our comprehensive design, showcasing the carefully planned routes and junction configurations. This visual representation is critical for understanding how traffic will navigate through the modeled area, allowing us to simulate and analyse various traffic scenarios effectively. The attention to detail in this design phase sets the foundation for robust simulations, ensuring that our models will yield valuable insights into traffic dynamics and potential improvements for the intersection management system.

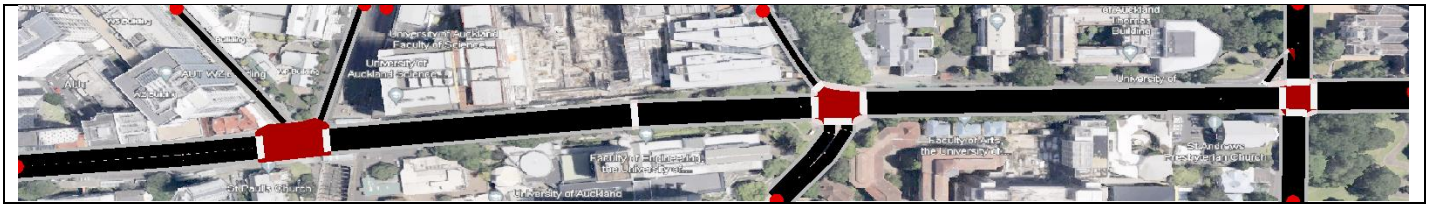


Figure 3: Initial model version

Further refinement of the design was necessary to enhance the accuracy and functionality of the simulation. During the review process, certain lanes were identified that, while included in the initial design, did not contribute to the overall simulation objectives. These unnecessary lanes led to errors due to overlaps between nodes and edges, which could compromise the integrity of the model.

Additionally, some of these extraneous edges interfered with pedestrian access pathways, particularly along sidewalks, posing a potential issue for accurately simulating pedestrian movements and their interactions with vehicle traffic. To resolve these challenges, the decision was made to remove the unnecessary edges and lanes from the design.

This adjustment not only eliminated the errors but also streamlined the model, ensuring that it accurately reflected the intended traffic flows and pedestrian access routes. By focusing on essential elements, the overall clarity and functionality of the simulation improved, setting the stage for more precise and effective analysis in subsequent phases. The final model, looked like the following.

Figure 4: Final model version

the next step was to employ traffic lights, pedestrian demands, and vehicle demands to create a simple simulation to check if the design is working and check for errors in the junction.

The process of adding traffic lights involved selecting a junction and utilizing the traffic light mode to auto-generate a traffic light configuration based on predefined route definitions. These route definitions specify which paths are permitted or restricted for vehicle movement, ensuring that demands can navigate through these routes without encountering conflicts. The traffic signal phases were designed to operate on a 90-second cycle, which could be easily modified to accommodate varying traffic conditions.

Phase logic is defined using a system of letters that correspond to the various traffic light colours. For example, in a phase represented by the sequence "GGrrgGGrr," the letters indicate the following: lanes designated with ‘G’ represent

major green lights, allowing for the uninterrupted flow of traffic; lanes marked with ‘g’ indicate minor green lights, permitting limited traffic movement from less-travelled roads; and lanes labelled ‘r’ denote red lights, signalling vehicles to stop. Additionally, the yellow light, which serves as a cautionary signal, is represented by the letter ‘y’. This systematic approach to phase logic ensures clear communication of traffic signal states, facilitating safe and efficient traffic management at intersections.

Below is an example of the traffic light mode looked like.

Figure 5: Traffic light mode

The next step involved creating traffic demands within the simulation for both vehicles and pedestrians. Demands are established through the demands tab, where users can choose between vehicle mode and pedestrian mode to generate the respective traffic flows. For vehicle generation, users can select the specific path vehicles will take and generate either individual vehicles or a continuous flow of traffic. Additionally, the aesthetics of the vehicles, such as colour and size, can be customized to enhance the simulation's visual appeal. Data from Auckland Transport website was used to generate vehicle demands. We used the data determining real-time paths and vehicle numbers. However, AT has a high volume of vehicle numbers per path which crashes the simulation. Therefore, the decision was made to reduce the number by a factor of 20 to allow the system to flow.

In contrast, generating pedestrian demands requires the prior establishment of sidewalks and designated walking areas within the network mode. Once these prerequisites are met, the pedestrian mode can be utilized to select the paths pedestrians should follow. This allows for the generation of either individual pedestrians or a flowing group, ensuring realistic pedestrian movement patterns in the simulation. Crossing was also added that allowed pedestrians to cross the street in a safe manner either with traffic light controlled or crossing priority rule. This comprehensive approach to demand creation ensures that both vehicular and pedestrian traffic is accurately represented, facilitating a more effective analysis of traffic flow dynamics at intersections.

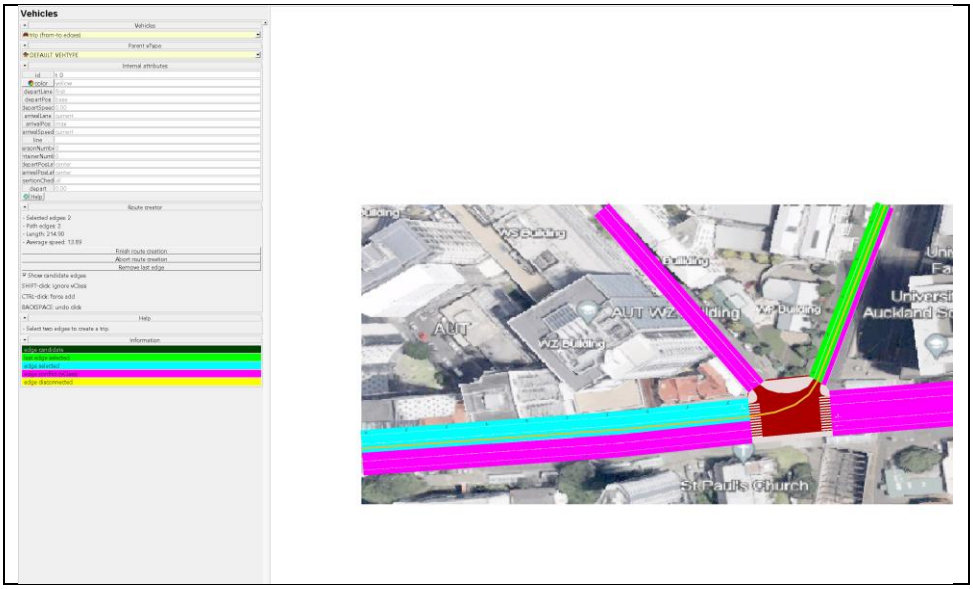


Figure 6: Vehicle Generation

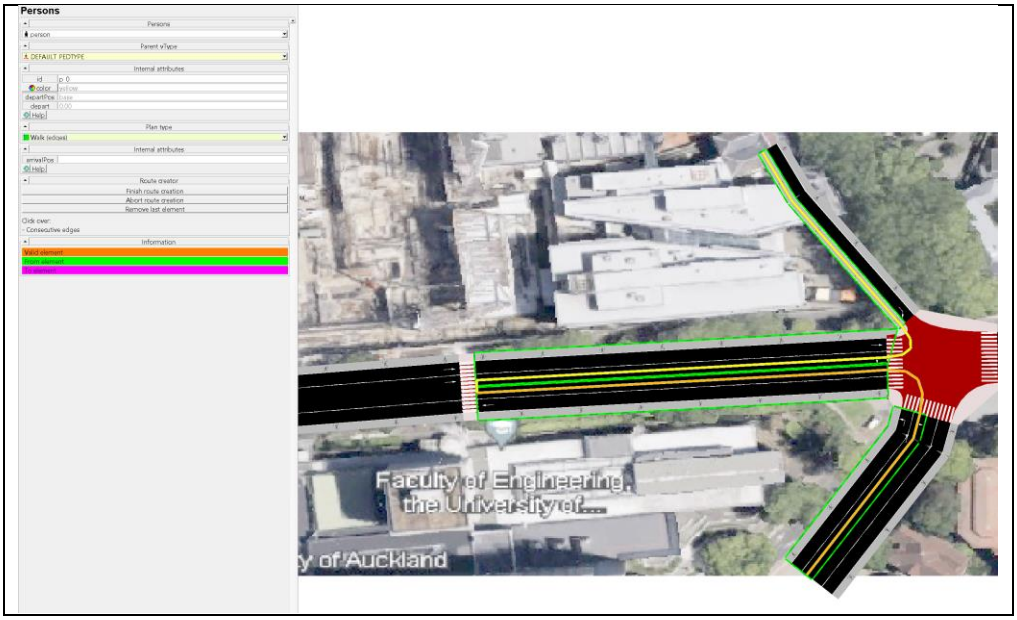


Figure 7: Person Generation

Once the vehicle and pedestrian demands were established, a preliminary simulation was conducted to gain a deeper understanding of the SUMO environment. This initial run provided valuable insights into the functionality of the software, allowing for the observation of traffic dynamics and interactions between vehicles and pedestrians. By analysing the outcomes of this simple simulation, further adjustments could be made to the traffic demands and configurations, enhancing the overall accuracy and effectiveness of the simulation. This foundational experience also facilitated a more comprehensive exploration of the capabilities and features within SUMO, setting the stage for more complex simulations in subsequent phases of the project.

TRACI Script Development

Following the initial simulation, the next step involved integrating a Traffic Control Interface (TraCI) server by developing various Python scripts. TraCI serves as a crucial communication interface between the SUMO simulation and external control programs, enabling real-time manipulation of traffic signals based on dynamic traffic conditions. Before the scripts could be developed, induction loop detectors were incorporated into the design using the NETEDIT tool. For the sake of the simulation, incorporating Digi Xbee modules, OBU's, V2X enabled devices and RSU's were proven difficult and therefore the assumption that these components can come under a single classification of induction loop detectors reduced the complexity of the project.

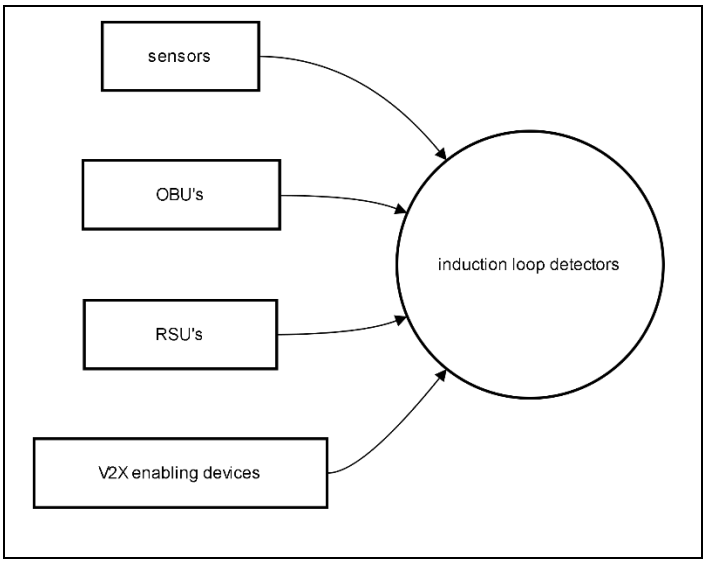


Figure 8: Detector Classification

These detectors collect data on vehicle flow and pedestrian data, providing essential input for the traffic management system. Once integrated, the Python scripts will initiate the simulation and utilize the data gathered from the induction loop detectors. This real-time data collection allows for informed adjustments to the traffic signal phases, enabling the system to respond effectively to current traffic conditions. By leveraging the capabilities of the TraCI server, the project aims to create a more adaptive and efficient traffic control system, optimizing traffic flow and improving safety at intersections. The TRACI script will act as the CTMS, that will gather data from the detectors and change the phases of the TLCS at each of the four junctions. Below goes through sections of the code that allowed achievement of traffic lights adapting to pedestrian and vehicle demands.

```
from __future__ import absolute_import
from __future__ import print_function

import os
import sys
import optparse
import random
```

This section imports necessary libraries and modules. The __future__ imports ensure compatibility for certain functions between Python versions. Libraries like os, sys, and optparse are standard libraries for handling system operations, command-line options, and random number generation.

```python
# we need to import python modules from the $SUMO_HOME/tools directory
if 'SUMO_HOME' in os.environ:
    tools = os.path.join(os.environ['SUMO_HOME'], 'tools')
    sys.path.append(tools)
else:
    sys.exit("please declare environment variable 'SUMO_HOME'")

from sumolib import checkBinary  # noqa
import traci  # noqa
```

This section checks for the environment variable SUMO_HOME, which should point to the installation directory of SUMO. It appends the tools directory to the Python path for importing SUMO's Python modules. If SUMO_HOME is not set, the script exits with an error message. The sumolib module is imported to check the binary of SUMO, and traci is imported to facilitate communication with the simulation.

```python
traffic_threshold1 = 3
traffic_threshold2 = 3
pedestrian_threshold = 5
pedestrian_waiting_time_threshold = 10  # Time in seconds


# Store vehicle arrival and departure times
vehicle_times = {}
```

Here, various threshold values are defined for traffic and pedestrian conditions. These thresholds dictate when to change traffic light phases based on vehicle and pedestrian counts. An empty dictionary, vehicle_times, is initialized to store the arrival and departure times of vehicles

```python
# Function to calculate dynamic green light duration based on vehicle count
def calculate_dynamic_duration(traffic_count):
    # Base duration
    base_duration = 20
    # Increase duration based on vehicle count
    return base_duration + min(traffic_count * 3, 30)  # Max increase to 30 seconds
```

This function computes the duration for the green light based on the number of vehicles detected at the traffic light. It starts with a base duration (20 seconds) and increases it by a factor related to the number of vehicles, with a maximum increase of 30 seconds.

```python
# Constants
MIN_GREEN_TIME = 15
VEHICLE_GREEN_PHASE = 0
PEDESTRIAN_GREEN_PHASES = {
    'J1': 8,  # For J1, pedestrian green phase is 8
    'J2': 3,  # For J2, pedestrian green phase is 3
    'J3': 10,  # For J3, pedestrian green phase is 10
    'J4': 9,  # For J4, pedestrian green phase is 9
}
TLSIDS = ['J1', 'J2', 'J3', 'J4']  # Add more traffic light IDs as needed

# Mapping of traffic light IDs to their corresponding pedestrian edges and crossings
PEDESTRIAN_MAPPING = {
    'J1': {
        'WALKINGAREAS': [':J5_w3', ':J5_w1', ':J5_w0'],
        'CROSSINGS': [':J5_c1', ':J5_c0']
    },
    'J2': {
        'WALKINGAREAS': [':J5_c0', ':J4_w0'],
        'CROSSINGS': [':J4_c0']
    },
    'J3': {
        'WALKINGAREAS': [':J3_w2', ':J3_w3', ':J3_w0', ':J3_w1'],
        'CROSSINGS': [':J3_c1', ':J3_c2', ':J3_c3']
    },
    'J4': {
        'WALKINGAREAS': [':J10_w2', ':J10_w3', ':J10_w0', ':J10_w1'],
        'CROSSINGS': [':J10_c2', ':J10_c1', ':J10_c0', ':J10_c3']
    },
}
```

MIN_GREEN_TIME: Minimum time (in simulation steps) that the vehicle green light should stay active.
VEHICLE_GREEN_PHASE: Represents the green phase for vehicles (phase 0).
PEDESTRIAN_GREEN_PHASES: A dictionary mapping each traffic light ID (TLSID) to the corresponding pedestrian green phase.
TLSIDS: A list of traffic light IDs that the system will control.
PEDESTRIAN_MAPPING: This dictionary defines the pedestrian walking areas and crossings for each traffic light. Each TLSID (e.g., 'J1', 'J2', etc.) has its own set of walking areas and crossings.

```
def checkWaitingPersons(tlsid):
    """Check whether a person has requested to cross the street at a given traffic light"""
    walking_areas = PEDESTRIAN_MAPPING[tlsid]['WALKINGAREAS']
    crossings = PEDESTRIAN_MAPPING[tlsid]['CROSSINGS']

    # Check both sides of the crossing
    for edge in walking_areas:
        peds = traci.edge.getLastStepPersonIDs(edge)
        for ped in peds:
            if (traci.person.getWaitingTime(ped) == 1 and
                traci.person.getNextEdge(ped) in crossings):
                num_waiting = traci.trafficlight.getServedPersonCount(tlsid, PEDESTRIAN_GREEN_PHASES[tlsid])
                print("%s: pedestrian %s pushes the button at %s (waiting: %s)" %
                      (traci.simulation.getTime(), ped, tlsid, num_waiting))
                return True
    return False
```

The pedestrian detection logic iterates through the walking areas associated with each traffic light (identified by tlsid). For each pedestrian in these areas, the script checks if they are waiting to cross by verifying if their waiting time equals 1, and whether their next destination is one of the designated crossings. If a pedestrian meeting these conditions is found, a message is printed to the console indicating that the pedestrian has "pushed the button" to cross. The function then returns True to signal that a pedestrian request has been made. If no such pedestrian is detected, the function returns False.

```
# Loop through each junction and its associated detectors
for junction, detectors in pedestrian_detectors.items():
    waiting_pedestrians = 0

    for detector in detectors:
        detected_pedestrians = traci.inductionloop.getLastStepPersonIDs(detector)

        for pedestrian_id in detected_pedestrians:
            # Check if pedestrian is already recorded
            if pedestrian_id not in pedestrian_times:
                pedestrian_times[pedestrian_id] = traci.simulation.getTime()

            # Calculate how long the pedestrian has been waiting
            waiting_time = traci.simulation.getTime() - pedestrian_times[pedestrian_id]

            # Check if waiting time exceeds the threshold
            if waiting_time >= pedestrian_waiting_time_threshold:
                waiting_pedestrians += 1
```

This part of the function loops through each junction and its detectors. For each detector, it checks for the IDs of pedestrians detected and calculates how long each pedestrian has been waiting. If a pedestrian has waited longer than the specified threshold, they are counted towards the total waiting pedestrians for that junction.

```
        # If enough pedestrians are waiting, trigger the traffic light for pedestrian crossing
        if waiting_pedestrians >= pedestrian_threshold:
            # Trigger the pedestrian crossing light (adjust the phase number based on your setup)
            if traci.trafficlight.getPhase("J1") != 8:                # Assuming phase 2 is for pedestrians
                traci.trafficlight.setPhase("J1", 8)  # Change to pedestrian green light phase
                traci.trafficlight.setPhaseDuration("J1", 15)  # Allow pedestrians to cross for 15 seconds

            if traci.trafficlight.getPhase("J2") != 3:
                traci.trafficlight.setPhase("J2", 3)
                traci.trafficlight.setPhaseDuration("J2", 15)

            if traci.trafficlight.getPhase("J3") != 3:
                traci.trafficlight.setPhase("J3", 3)
                traci.trafficlight.setPhaseDuration("J3", 15)

            if traci.trafficlight.getPhase("J4") != 3:
                traci.trafficlight.setPhase("J4", 3)
                traci.trafficlight.setPhaseDuration("J4", 15)
```

If the number of waiting pedestrians meets or exceeds the specified threshold, the corresponding traffic lights are changed to allow pedestrian crossing. The phase numbers used in set Phase may need to be adjusted based on the specific traffic light configuration.

```
def run():
    """execute the TraCI control loop"""
    step = 0
    traci.trafficlight.setPhase("J1", 0)
    traci.trafficlight.setPhase("J2", 0)
    traci.trafficlight.setPhase("J3", 0)
    traci.trafficlight.setPhase("J4", 0)

    while traci.simulation.getMinExpectedNumber() > 0:
        traci.simulationStep()

        # Get the list of vehicles currently in the simulation
        current_vehicles = traci.vehicle.getIDList()
```

This function executes the main loop of the TraCI control process. It initializes the traffic light phases for each junction and continues to loop as long as there are vehicles expected in the simulation. Each iteration represents a simulation step.

```
# Logic for traffic light J1
if traci.trafficlight.getPhase("J1") == 0:
    count = traci.inductionloop.getLastStepVehicleNumber("D1.1")
    if count > traffic_threshold1:
        duration = calculate_dynamic_duration(count)
        traci.trafficlight.setPhase("J1", 4)
        traci.trafficlight.setPhaseDuration("J1", duration)
        traci.trafficlight.setPhase("J1", 1)
        traci.trafficlight.setPhaseDuration("J1", 2)
    elif traci.inductionloop.getLastStepVehicleNumber("D1.3") > traffic_threshold1:
        duration = calculate_dynamic_duration(count)
        traci.trafficlight.setPhase("J1", 3)
        traci.trafficlight.setPhaseDuration("J1", duration)
        traci.trafficlight.setPhase("J1", 1)
        traci.trafficlight.setPhaseDuration("J1", 2)
    else:
        traci.trafficlight.setPhase("J1", 0)

# Logic for traffic light J2
if traci.trafficlight.getPhase("J2") == 0:
    count = traci.inductionloop.getLastStepVehicleNumber("D2.1")
    if count > traffic_threshold1:
        duration = calculate_dynamic_duration(count)
        traci.trafficlight.setPhase("J2", 2)
        traci.trafficlight.setPhaseDuration("J2", duration)
        traci.trafficlight.setPhase("J2", 1)
        traci.trafficlight.setPhaseDuration("J2", 2)
    else:
        traci.trafficlight.setPhase("J2", 0)
```

This segment defines the logic for managing traffic light phases for junctions J1, J2 but for J3, J4 the structure is the same. It checks the current phase, and the count of vehicles detected by induction loops. If the count exceeds the defined threshold, the traffic light phase is set to allow vehicles to pass, with a dynamic duration based on vehicle count.

```
        check_pedestrian_detectors()

        step += 1
```

At the end of each simulation step, the script checks for waiting pedestrians using the previously defined function check_pedestrian_detectors(), ensuring the system is responsive to pedestrian needs.

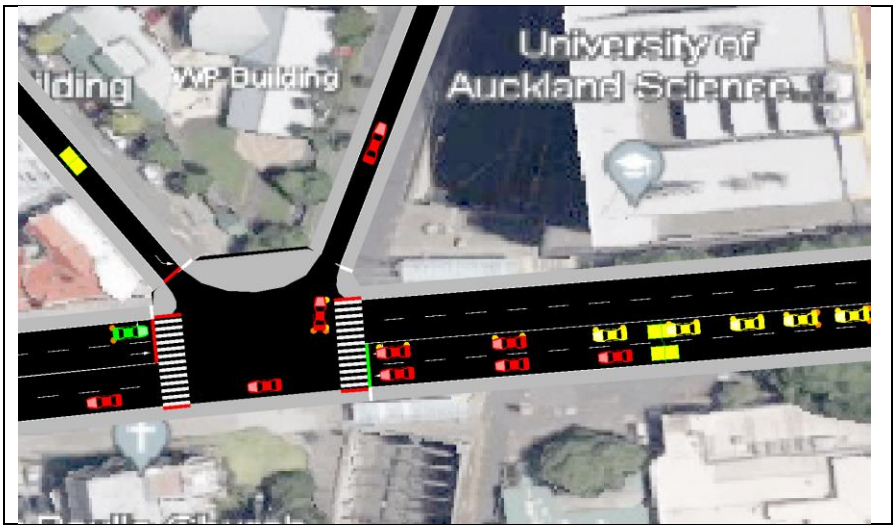
```
# Main entry point of the script
if __name__ == "__main__":
    options = get_options()

    # Start SUMO as a subprocess and connect
    if options.nogui:
        sumoBinary = checkBinary('sumo')
    else:
        sumoBinary = checkBinary('sumo-gui')

    traci.start([sumoBinary, "-c", "data/Project.sumocfg", "--tripinfo-output", "tripinfo.xml"])
    run()
```

This final section runs the script. It checks for the SUMO binary, starts the TraCI connection with the specified configuration file, and calls the run() function to initiate the simulation. After the simulation ends, it closes the TraCI connection properly.

Chapter 4: Results

The simulation of the script was executed to evaluate the operational status of the traffic light logic. Additionally, an analysis was conducted to verify whether the detectors successfully detected vehicle demands. Upon completion of the simulation, a trip information file was generated, providing valuable insights into the performance of the traffic management system and the effectiveness of the implemented logic. This output served as a critical resource for assessing the overall functionality and identifying any areas for improvement within the traffic control framework.

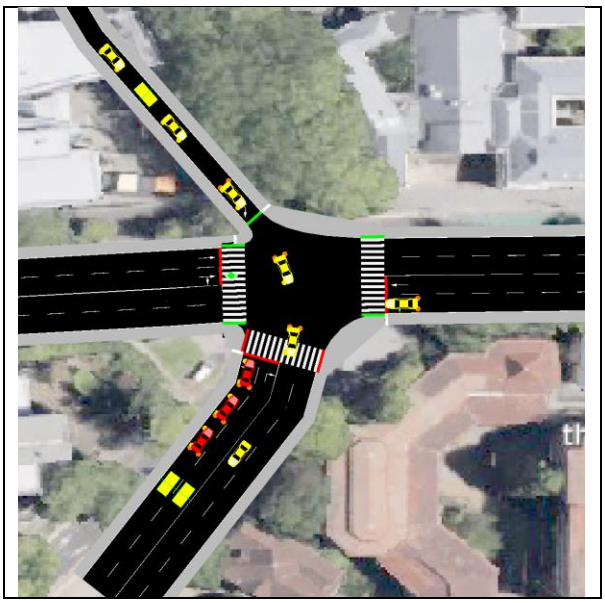
Scenario 1

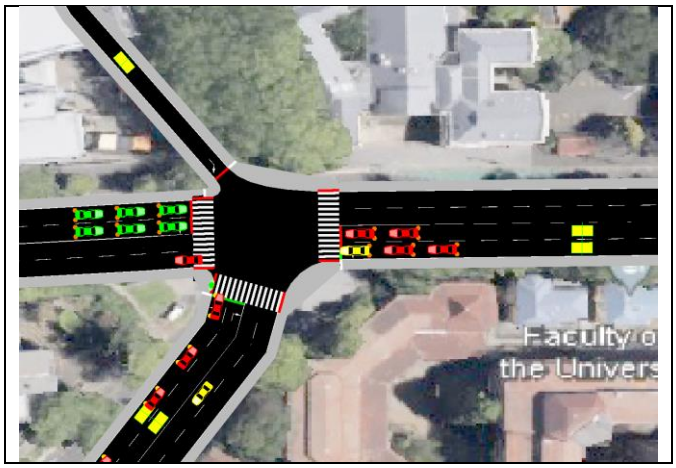


In this simulation scenario, the observations indicated that at Junction 1, the green phase for vehicles turning right was activated upon the detector confirming that the vehicle threshold had been reached. This critical functionality allowed for a timely transition to the green phase, enabling vehicles to proceed smoothly through the intersection.

The system effectively monitored the real-time traffic conditions, ensuring that the traffic lights adapted to the actual demand. Once the specified threshold of vehicles was detected, the logic executed a phase change that optimized traffic flow, minimizing delays for right-turning vehicles. This adaptive approach not only enhanced the efficiency of vehicle movement at the junction but also contributed to overall traffic management objectives by reducing congestion and improving the responsiveness of the traffic signal system. The simulation results provided essential data for further analysis, facilitating the ongoing refinement of the traffic light control strategy.

Scenario 2





At Junction 2, the simulation revealed a continuous flow of vehicles moving from north to south, as the detector consistently registered that the vehicle threshold was being met. This steady stream of traffic allowed the green phase for north-south movement to remain active, facilitating uninterrupted passage for those vehicles.

Once the last of the vehicles departed and the detector indicated that the number of vehicles had dropped below the threshold, the system promptly executed a phase change. This transition enabled vehicles in the opposing lane to proceed, ensuring efficient use of the traffic signal system. The adaptive nature of the traffic light control demonstrated its ability to respond to real-time conditions, optimizing traffic flow and minimizing unnecessary delays for all directions. The data gathered from this scenario provided valuable insights into the effectiveness of the detection and phase transition logic, contributing to the ongoing evaluation and enhancement of the traffic management strategy at Junction 2.

Scenario 3

During the simulation, it was observed that pedestrians waited at the traffic lights, signalling their intent to cross. The system effectively monitored pedestrian activity, ensuring their safety and right of way. Once the detector received a vehicle count indicating that the number of vehicles had fallen below the established threshold, it executed a phase change to allow the waiting pedestrians to proceed through the crossing.

Additionally, the system provided real-time feedback by outputting pedestrian input data to the console, clearly indicating the presence of pedestrians waiting to cross. This functionality not only enhanced the responsiveness of the traffic control system but also reinforced the commitment to pedestrian safety by prioritizing their movement at the appropriate times. The integration of pedestrian detection and signalling into the traffic management strategy demonstrated a comprehensive approach to managing both vehicle and pedestrian traffic, contributing to a more efficient and safe intersection environment.

Trip information and Detector data

Once the simulation was completed, a trip information file was generated, providing valuable insights into the performance of the traffic management system. By examining this file, key metrics such as the duration of vehicles at the intersection and the total time spent waiting were extracted.

To optimize the program further, a comparison was conducted between the outputs generated by the script and the overall simulation output. This analysis aimed to identify any discrepancies or changes in the data sets. However, upon reviewing the results, no differences were found between the two data sets. This outcome led to the conclusion that there may be a logical error within the code, or alternatively, that the output file was accurately reflecting the information captured by the detector data.

In addition to trip information, the simulation also recorded detector data upon completion, confirming that vehicles were being detected as intended. This functionality ensured that the detection system operated effectively throughout the simulation. Below is a representation of how the data sets for trip information and detector information appear:

Trip Information Data Set:

Vehicle ID: [1, 2, 3, ...]
Duration at Junction: [10s, 15s, 12s, ...]
Total Wait Time: [5s, 8s, 4s, ...]

Detector Information Data Set:

Time Stamp: [0s, 1s, 2s, ...]
Vehicle Count: [3, 5, 4, ...]
Pedestrian Count: [1, 2, 0, ...]

These data sets serve as essential resources for understanding the operational efficiency of the traffic control system and identifying areas for potential improvements.

Table 1: Trip info dataset

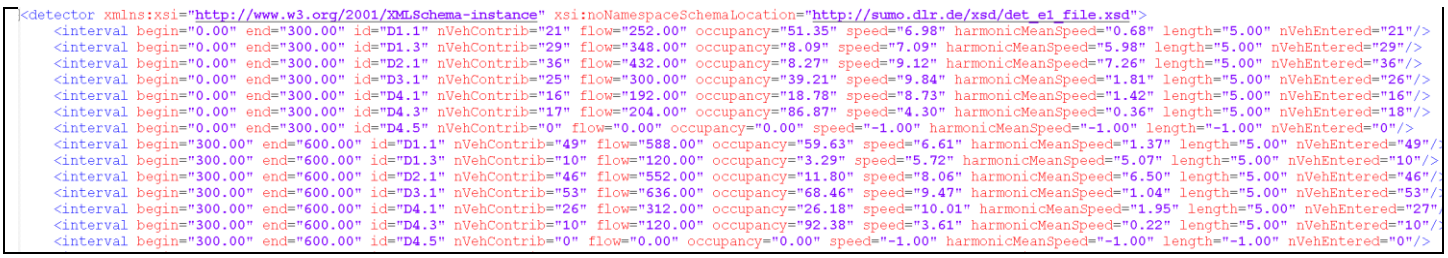


Table 2: Detector dataset

Chapter 5: Evaluation and Discussion

Logical Connection of Theoretical and Practical Project Aspects

The integration of theoretical concepts and practical applications in this project is anchored in the use of Vehicle-to-Everything (V2X) communication via IEEE 802.11p and the application of Digi XBee components. These technologies are well-documented in the literature for their ability to enhance safety and efficiency at intersections, and their deployment in the SUMO simulation provided a practical demonstration of how these systems can improve traffic flow. The theoretical foundation, supported by the case study of Singapore's GLIDE system, aligned well with the project's aims of reducing congestion and improving pedestrian and vehicle safety at intersections.

The conceptual design leveraged inductive loop detectors for real-time data collection, which was translated into the Python-based TRACI script for adaptive traffic light control. This ensured that the theoretical principles surrounding smart transportation systems were realized in a practical, operational model. The project also considered the expectations of key stakeholders, such as city planners and road users, by focusing on the development of a system that enhances real-time decision-making at intersections.

Judgement of Project Outcomes

The project outcomes were evaluated using key performance metrics such as vehicle wait times, total time spent at intersections, and system responsiveness to both traffic and pedestrian demands. The TRACI script's ability to dynamically adjust traffic light phases in real-time, based on inputs from vehicle and pedestrian detectors, marked a significant improvement over traditional fixed-timing traffic systems. This adaptive approach offered the potential for more efficient traffic management, reducing delays and improving safety.

However, while the simulation did demonstrate notable improvements in traffic flow compared to the base model, some challenges arose in detecting substantial differences across datasets with varying threshold conditions. This lack of variation pointed to potential logic errors within the TRACI script, indicating that further refinement is necessary to enhance the system's performance. Despite these challenges, the project effectively recorded accurate detector data, confirming that both vehicles and pedestrians were reliably detected. The generation of trip information files further validated the system's capability to track key performance indicators, enabling a thorough evaluation of the traffic management system across different scenarios.

In reviewing the project's outcomes against its original objectives, several significant achievements were made. The first objective—designing and modeling traffic flow within the SUMO environment—was successfully completed for the selected area of Symonds Street, from the AUT-WZ building to Anzac Avenue. The simulation utilized real-world data from Auckland Transport (AT) to design vehicle routes, ensuring that the traffic dynamics of the urban corridor were accurately represented. This provided a realistic platform for testing and refining traffic light management strategies.

The second objective—developing a Python-based TRACI script to control traffic lights dynamically—was also accomplished. The script enabled real-time control of traffic light phases in response to both vehicle and pedestrian data, offering a dynamic alternative to conventional traffic management systems. However, as previously noted, further refinements are needed, particularly in the handling of logic, to improve the integration of vehicle and pedestrian detection. By addressing these issues, the system can evolve into a more robust and effective solution for managing complex traffic scenarios.

Finally, the third objective, which aimed to provide insights into the integration of autonomous vehicles, was successfully achieved through the simulation of traffic demands within SUMO. By incorporating autonomous vehicles into the system and utilizing the TRACI script to manage traffic lights, the project provided valuable insights into how autonomous vehicles could seamlessly interact with existing traffic infrastructure. This sets a foundation for future real-world applications, helping pave the way for the smooth integration of autonomous vehicles into New Zealand's transportation network.

Demonstration of Critical Thinking

The project exhibited critical thinking through the iterative development and troubleshooting of the system. Several challenges arose during the SUMO model development, such as issues with pedestrian flow and sidewalk configurations. The decision to simplify vehicle detection by classifying multiple components under induction loop detectors showcased problem-solving under project constraints. Additionally, the identification of logic errors in the TRACI script indicated a need for deeper analysis of the code and a more nuanced understanding of SUMO's simulation environment.

By acknowledging these challenges and implementing adjustments, the project demonstrated a proactive approach to overcoming limitations. Moreover, the identification of research gaps—such as the need for improved scalability, standardization, and cybersecurity in smart transportation systems—provided a critical evaluation of the current state of technology, highlighting areas for future research and development.

This systematic evaluation of both the successes and shortcomings of the project underscores its potential while pointing toward the necessity of continued refinement to achieve greater real-world applicability.

Project Challenges

Throughout the modelling and development phases, several significant challenges were encountered. Initially, the manual design process using SUMO's XML files was labour-intensive and error prone. Editing lines directly within these XML files to construct the system was tedious, making both system creation and error-checking time-consuming and difficult. Shifting to NETEDIT greatly streamlined the design process, but it introduced other challenges. Since many of NETEDIT's default settings are based on U.S. standards, such as right-side driving rules, additional configuration was required to adapt the system to New Zealand's left-hand traffic regulations. The process of generating sidewalks and pedestrian routes also presented difficulties, as NETEDIT only allowed pedestrian movement in one direction along sidewalks. This limitation restricted the flexibility needed to model realistic pedestrian behaviour. Additionally, generating safe pedestrian crossings was problematic, as NETEDIT often failed to produce crossings, even when edge conditions were met. Slip roads caused further issues by interfering with sidewalk generation, pedestrian walking areas, and crosswalk configurations, leading to irregular junctions and additional time spent reconfiguring traffic lights.

The development of the TRACI Python script proved to be the most difficult aspect of the project. Understanding how the TRACI server functioned required upskilling in Python programming, as the learning curve was steep. Although tutorials in the SUMO_Home directory provided a solid foundation, the Python language's sensitivity to indentation caused frequent issues. Often, simulations would fail to open due to minor indentation errors, necessitating detailed line-by-line code reviews to identify and fix problems. Logical errors within the script were also common, frequently causing the TRACI server to disconnect. Combining the detection logic for both vehicles and pedestrians in the same script proved challenging. To address this, the solution involved splitting the functionality into two separate scripts—one focused on detecting vehicles via induction loop detectors and changing traffic phases, while the other handled pedestrian detection and phase adjustments based on waiting duration thresholds. Reviewing and refining the script will give a more cohesive approach in combing the two logics together to have a single script that operates based on pedestrian and vehicle changes simultaneously. Despite these obstacles, the adjustments allowed for continued progress in developing an adaptive traffic light system.

Chapter 6: Conclusion

This project successfully established a foundational framework for a smart transportation system aimed at improving traffic flow and safety at urban intersections, with future applications for autonomous vehicles in New Zealand. By integrating theoretical models such as Vehicle-to-Everything (V2X) communication via IEEE 802.11p and practical tools like the SUMO traffic simulation environment, the project achieved a significant milestone in designing a dynamic traffic management system.

The project's main goal was to create an adaptive traffic light control system capable of optimizing traffic flow, reducing congestion, and enhancing safety for both vehicles and pedestrians. This goal was largely met through the development of the Python-based TRACI script, which dynamically adjusted traffic light phases in response to real-time data collected from vehicle and pedestrian detectors. The simulation demonstrated that the system was effective in adapting traffic lights to actual demand, showing improvements over traditional fixed-timing systems. The success criteria, including the reduction of vehicle congestion and the ability to handle real-time pedestrian crossings, were achieved.

However, the project encountered some limitations, particularly in the integration of pedestrian and vehicle detection within a single script. The separate handling of vehicle and pedestrian logic, while functional, highlighted the need for a more unified approach. Moreover, despite accurate data collection from detectors, a lack of significant variation in output under different simulation conditions suggested potential logic issues within the TRACI script, warranting further optimization.

Achievement of Project Goals

The project's core objectives of dynamically managing traffic light phases and improving intersection safety were successfully achieved, showcasing the potential for smart transportation systems to manage the complexity of urban mobility. This accomplishment is particularly timely as New Zealand prepares for the integration of autonomous vehicles into its transport infrastructure. By simulating traffic conditions in a controlled environment and managing real-time demands from both vehicles and pedestrians, the project demonstrated how adaptive systems can significantly enhance traffic efficiency and safety.

Recommendations for Future Work

While this project has laid a strong foundation, several areas for future research and development remain. To enhance the robustness and applicability of the smart transportation system, the following steps are recommended:

Analysis of More Complex Intersections: Future work should involve the simulation of more complex intersections, such as multi-lane roads, roundabouts, and intersections with bus lanes, to test the system's scalability and adaptability. These more complex setups will provide insights into how the system handles varying levels of congestion and diverse traffic patterns.

Integration of Multiple Transport Modes: The current system focuses primarily on standard vehicle and pedestrian interactions. Future iterations should include additional forms of transportation such as buses, taxis, bikes, and emergency vehicles. This will make the system more reflective of real-world urban traffic, where multiple types of vehicles share the road and have varying priority levels.

Refinement of the TRACI Script: Further refinement of the Python-based TRACI script is essential for improving the system's responsiveness to traffic and pedestrian demands. Optimizing the code will reduce potential logic errors and enhance its ability to handle different traffic scenarios, thresholds, and demand levels more effectively. A key area of improvement is the merging of the current pedestrian and vehicle detection scripts. By combining these into a single, integrated solution, the Central Traffic Management System (CTMS) will be able to adapt more fluidly to real-time inputs from both sources. This will streamline the system's functionality and increase its efficiency in managing simultaneous demands.

Physical Model or Real-World Testing: To move beyond simulation, real-world or physical model testing is necessary to validate the system's effectiveness in practical applications. Testing the system in a controlled, yet real-world environment would provide invaluable feedback on its performance, reliability, and scalability.

Simulation with Variable Speeds and Dynamic Features: Future work should incorporate variable vehicle speeds, lane changes, and other dynamic traffic features to make the simulation more realistic. This will help ensure that the system performs well under diverse and unpredictable real-world conditions.

Utilization of Artificial Intelligence (AI): AI can play a transformative role in enhancing the system's decision-making capabilities. By implementing AI algorithms, the traffic management system could become more predictive and responsive, learning from traffic patterns and adjusting signal timings to optimize flow without human intervention.

Addressing Security and Sustainability: As smart transportation systems rely heavily on data exchange and connectivity, ensuring robust cybersecurity is crucial. Future developments should focus on creating secure protocols that protect the system from cyber threats while maintaining its functionality. Additionally, considerations for the sustainability of the system, such as energy efficiency and long-term maintenance, should be explored.

Long-Term Impact and Applicability

This project not only serves as a stepping stone for the development of smart transportation systems in New Zealand but also aligns with the global trend toward autonomous vehicles and intelligent traffic management. The simulation and development work conducted here provide a solid foundation for the next generation of urban mobility solutions, particularly as cities seek to reduce congestion, improve road safety, and integrate autonomous vehicles into their infrastructure. By continuously refining and expanding upon this framework, the system has the potential to be implemented in real-world traffic environments, contributing to safer, more efficient, and sustainable urban transportation networks.

In conclusion, while this project achieved its primary objectives, further research and development are essential to fully realize the potential of a smart transportation system that can handle the complexity of real-world urban mobility. By addressing the recommended areas for improvement, future iterations of this system can become an integral part of the transportation infrastructure in New Zealand and beyond, paving the way for safer and more efficient cities.

Appendices

```
Initialize SUMO environment and dependencies
Set thresholds for vehicle and pedestrian detection
Initialize global variables for tracking vehicle and pedestrian times

Define function to calculate green light duration based on vehicle count:
  - Base duration is 20 seconds
  - Add extra time based on vehicle count, capped at 30 seconds

Start the simulation:
  - Set initial traffic light phases for intersections J1, J2, J3, and J4 to red (0)

While the simulation is running:
  - Step through the simulation one step at a time

  For traffic light J1:
    - If traffic light is red (phase 0):
      - Check the number of vehicles on loop detector D1.1
      - If vehicle count exceeds threshold:
        - Set green light for J1 (phase 4) with calculated duration
      - Else check for vehicles on loop D1.3
      - If vehicle count exceeds threshold:
        - Set green light for J1 (phase 3) with calculated duration
      - Else keep the light red

  For traffic light J3:
    - If traffic light is red (phase 0):
      - Check the number of vehicles on loop detector D3.3
      - If vehicle count exceeds threshold:
        - Set green light for J3 (phase 6) with calculated duration
      - Else check for vehicles on loops D3.6 and D3.2
      - Set appropriate green light phases based on vehicle count
      - Else keep the light red

  For traffic light J4:
    - If traffic light is red (phase 0):
      - Check vehicle counts on detectors D3.2 and D3.5
      - If vehicle count exceeds threshold:
        - Set green light for J4 (appropriate phase) with calculated duration
      - Else keep the light red

  - Continue stepping through the simulation

End the simulation
Close SUMO connection
```

Figure 9: Vehicle Pseudo code

```
Initialize SUMO environment and dependencies
Set constants for minimum green time and traffic light phases
Define mappings for traffic light IDs, pedestrian areas, and crossings

Define function to execute the simulation control loop:
  - Initialize green time tracking for each traffic light
  - Initialize active request tracking for pedestrian crossings

  While there are vehicles in the simulation:
    - Step through the simulation

    For each traffic light ID (TLSID):
      If no active pedestrian requests for this traffic light:
        Check if a pedestrian has requested to cross

      If traffic light is green for vehicles:
        Increment green time counter for this traffic light
        If green time exceeds minimum threshold:
          If there are active pedestrian requests:
            - Change traffic light to pedestrian green phase
            - Reset state for pedestrian requests
            - Reset green time counter

  Close the simulation connection

Define function to check for waiting pedestrians at a traffic light:
  - For each walking area associated with the traffic light:
    - Get the list of pedestrians currently in the area
    For each pedestrian:
      If waiting time indicates they want to cross:
        - Log the request and return true
  Return false if no requests are found

Define function to get command line options

Main entry point of the script:
  - Parse command line options
  - Start SUMO simulation with specified configuration and output files
  - Execute the control loop
```

Figure 10: Pedestrian Pseudo code

References

References
[1] J. Yan, J. Liu, and F.-M. Tseng, “An evaluation system based on the self-organizing system framework of smart cities: A case study of smart transportation systems in China,” Technological Forecasting and Social Change, vol. 153, p. 119371, Apr. 2020, doi: 10.1016/j.techfore.2018.07.009.

[2] P. Davidson and A. Spinoulas, “Autonomous Vehicles - What Could This Mean For The Future Of Transport?,” 2015.

[3] “Designing a Smart Transportation System: An Internet of Things and Big Data Approach.” https://ieeexplore.ieee.org/abstract/document/8809663/?casa_token=0jGXO5YZkjwAAAAA:43rKjNgc9yTKKEo2xhO59dmE85CvR8HDvcyCWJe7BXXE1vbUaCQPuKswRqfpelzvwJFEpsGa1llc (accessed Sep. 11, 2023).

[4] M. Derawi, Y. Dalveren, and F. A. Cheikh, “Internet-of-Things-Based Smart Transportation Systems for Safer Roads,” in 2020 IEEE 6th World Forum on Internet of Things (WF-IoT), Jun. 2020, pp. 1–4. doi: 10.1109/WF-IoT48130.2020.9221208.

[5] J. Guerrero-Ibáñez, S. Zeadally, and J. Contreras-Castillo, “Sensor Technologies for Intelligent Transportation Systems,” Sensors, vol. 18, no. 4, Art. no. 4, Apr. 2018, doi: 10.3390/s18041212.

[6] “Survey of various data collection ways for smart transportation domain of smart city.” https://ieeexplore.ieee.org/abstract/document/8058265/ (accessed Sep. 11, 2023).

[7] Q. Cui, H. Kan, J. Zhao, J. Shen, and K. Xue, “Current status of smart city researches in China using the co-word analysis method,” in 4th International Conference on Smart and Sustainable City (ICSSC 2017), Jun. 2017, pp. 1–6. doi: 10.1049/cp.2017.0129.

[8] B. Kurniawan, S. Supangkat, F. Lumban Gaol, and B. Ranti, “Framework for Developing Smart City Models in Indonesian Cities (Based On Garuda Smart City Framework),” in 2022 International Conference on ICT for Smart Society (ICISS), Aug. 2022, pp. 01–05. doi: 10.1109/ICISS55894.2022.9915073.

[9] G. C. Heck, R. Hexsel, V. B. Gomes, L. Iantorno, L. L. Junior, and T. Santana, “GRID-CITY: A Framework to Share Smart Grids Communication with Smart City Applications,” in 2021 IEEE International Smart Cities Conference (ISC2), Sep. 2021, pp. 1–4. doi: 10.1109/ISC253183.2021.9562794.

[10] “Smart governance: A key factor for smart cities implementation | IEEE Conference Publication | IEEE Xplore.” Accessed: Oct. 03, 2023. [Online]. Available: https://ieeexplore.ieee.org/document/8038591

[11] “Smart World Living Lab: A Living Lab Approach to Improve Smart City Implementation (Introducing the DDG areas as an integrated Living Lab) | IEEE Conference Publication | IEEE Xplore.” Accessed: Oct. 03, 2023. [Online]. Available: https://ieeexplore.ieee.org/document/9922556

[12] A. Sahba and R. Sahba, “An Intelligent System for Safely Managing Traffic Flow of Connected Autonomous Vehicles at Multilane Intersections in Smart Cities,” in 2022 IEEE 12th Annual Computing and Communication Workshop and Conference (CCWC), Jan. 2022, pp. 0625–0631. doi: 10.1109/CCWC54503.2022.9720878.

[13] “Development Of Models Of Smart Intersections In Urban Areas Based On IoT Technologies | IEEE Conference Publication | IEEE Xplore.” Accessed: Oct. 03, 2023. [Online]. Available: https://ieeexplore.ieee.org/document/9751263

[14] “Development Of Models Of Smart Intersections In Urban Areas Based On IoT Technologies | IEEE Conference Publication | IEEE Xplore.” Accessed: Oct. 03, 2023. [Online]. Available: https://ieeexplore.ieee.org/document/9751263

[15] “Intersection-based Distance and Traffic-Aware Routing (IDTAR) protocol for smart vehicular communication | IEEE Conference Publication | IEEE Xplore.” Accessed: Oct. 03, 2023. [Online]. Available: https://ieeexplore.ieee.org/document/7986334

[16] L. Brown et al., “Smart Intersections: A Pathway to Resilient and Sustainable Cities,” in 2023 Smart City Symposium Prague (SCSP), May 2023, pp. 1–7. doi: 10.1109/SCSP58044.2023.10146117.

[17] “Develop A Virtual System Using Traffic Flow Model For Intelligent Transportation in Smart City | IEEE Conference Publication | IEEE Xplore.” Accessed: Oct. 03, 2023. [Online]. Available: https://ieeexplore.ieee.org/document/8587953

[18] M. Derawi, Y. Dalveren, and F. A. Cheikh, “Internet-of-Things-Based Smart Transportation Systems for Safer Roads,” in 2020 IEEE 6th World Forum on Internet of Things (WF-IoT), Jun. 2020, pp. 1–4. doi: 10.1109/WFIoT48130.2020.9221208.

[19] J. Qiao, “Smart City and Intelligent Upgrading of Urban Transportation System: based on Sustainable Investment Strategy,” in 2022 Second International Conference on Advanced Technologies in Intelligent Control, Environment, Computing & Communication Engineering (ICATIECE), Dec. 2022, pp. 1–6. doi: 10.1109/ICATIECE56365.2022.10047675.

[20] J. Meiying, L. Hainana, and N. Led, “The Evaluation Studies of Regional Transportation Accessibility Based on Intelligent Transportation System: Take the Example in Yunnan Province of China,” in 2015 International Conference on Intelligent Transportation, Big Data and Smart City, Dec. 2015, pp. 862–865. doi: 10.1109/ICITBS.2015.218.

[21] “Traffic Signaling Optimization for Intelligent and Green Transportation in Smart Cities | IEEE Conference Publication | IEEE Xplore.” Accessed: Oct. 03, 2023. [Online]. Available: https://ieeexplore.ieee.org/document/8537325

[22] “Dynamic Spatial Area Design for Transportation Management at Smart Road Lighting Platform System in Korea | IEEE Conference Publication | IEEE Xplore.” Accessed: Oct. 03, 2023. [Online]. Available: https://ieeexplore.ieee.org/document/9620787